Appendix

Locally Normalized Proposal. For Estimator 1, we instantiate the proposal distribution Q by locally normalizing the outgoing weights of W. For each node $x \in V$, define $d_W(x) = \sum_{y \in V} W_{xy}, \quad N_W^+(x) = \{y \in V : W_{xy} > 0\}$.

If $d_W(x) = 0$, then node x has no outgoing positive-weight transition, and no proposal distribution is required. A sampled prefix reaching such a node before completing the required length is terminated with zero contribution. In particular, if $d_W(v) = 0$, then $c_r(v) = 0$ for all $r \geq 1$. For every node x with $d_W(x) > 0$, we define

$$Q(x, y) = \begin{cases} \dfrac{W_{xy}}{d_W(x)}, & y \in N_W^+(x), \\ 0, & y \notin N_W^+(x). \end{cases}$$

Thus, $Q(x, y) > 0$ whenever $W_{xy} > 0$. Hence, every positive-weight transition under W can be sampled by the proposal distribution, and transitions with larger diffusion weights are sampled with larger probability. Under this locally normalized proposal, the single-sample estimator in Eq. (2) simplifies to

$$X_r(v; \pi) = \left(\prod_{i=0}^{r-2} d_W(v_i) \right) W_{v_{r-1} v}.$$

The final factor $W_{v_{r-1} v}$ checks whether the sampled prefix can return to the root node v. If no such return edge exists, then $W_{v_{r-1} v} = 0$, and the sample contributes zero.

Proof (Proof of Lemma 1). Estimator 1 uses the locally normalized proposal $Q(x, y) = W_{xy}/d_W(x)$ for every x with $d_W(x) > 0$ and every y such that $W_{xy} > 0$. If $d_W(v) = 0$, then $c_r(v) = 0$ and the claim is trivial. Otherwise, for a sampled prefix $\pi = (v_0, v_1, \ldots, v_{r-1})$ with $v_0 = v$, define $a_r(v; \pi) = \left(\prod_{i=0}^{r-2} W_{v_i v_{i+1}} \right) W_{v_{r-1} v}$. By definition, Estimator 1 uses $X_r(v; \pi) = a_r(v; \pi)/p_Q(\pi)$ whenever $p_Q(\pi) > 0$, and assigns zero contribution to terminated samples. We first prove unbiasedness. Taking expectation over the sampled prefix gives

$$\mathbb{E}[X_r(v; \pi)] = \sum_{\pi : p_Q(\pi) > 0} p_Q(\pi) \frac{a_r(v; \pi)}{p_Q(\pi)} = \sum_{\pi : p_Q(\pi) > 0} a_r(v; \pi).$$

Every positive-weight transition under W has positive probability under Q. Therefore, every positive-weight prefix contributing to $c_r(v)$ has nonzero probability under Q, and

$$\begin{aligned} \sum_{\pi : p_Q(\pi) > 0} a_r(v; \pi) &= \sum_{v_1, \ldots, v_{r-1} \in V} W_{v v_1} W_{v_1 v_2} \cdots W_{v_{r-2} v_{r-1}} W_{v_{r-1} v} \\ &= (W^r)_{vv} = c_r(v). \end{aligned}$$

Thus, $\mathbb{E}[X_r(v; \pi)] = c_r(v)$. By linearity of expectation over s independent samples, $\mathbb{E}[\widehat{c}_r(v)] = \frac{1}{s} \sum_{j=1}^{s} \mathbb{E}[X_r(v; \pi^{(j)})] = c_r(v)$. It remains to compute the

variance. The second moment of the single-sample estimator is

$$\mathbb{E}[X_r(v;\pi)^2] = \sum_{\pi:p_Q(\pi)>0} p_Q(\pi)\left(\frac{a_r(v;\pi)}{p_Q(\pi)}\right)^2 = \sum_{\pi:p_Q(\pi)>0} \frac{a_r(v;\pi)^2}{p_Q(\pi)}.$$

Since $\mathbb{E}[X_r(v;\pi)] = c_r(v)$, we obtain $\mathrm{Var}(X_r(v;\pi)) = \mathbb{E}[X_r(v;\pi)^2] - \mathbb{E}[X_r(v;\pi)]^2 = \sum_{\pi:p_Q(\pi)>0} \frac{a_r(v;\pi)^2}{p_Q(\pi)} - c_r(v)^2$. Finally, because $\widehat{c}_r(v) = \frac{1}{s}\sum_{j=1}^{s} X_r(v;\pi^{(j)})$ is the average of s independent copies of $X_r(v;\pi)$, we have $\mathrm{Var}(\widehat{c}_r(v)) = \frac{1}{s}\mathrm{Var}(X_r(v;\pi))$. Substituting the single-sample variance gives the claimed result.

The next lemma establishes the unbiasedness and variance of the HISP-Vec estimator. It shows that HISP-Vec is unbiased for the diagonal statistic $c_r(v) = (W^r)_{vv}$, while its variance is determined by the off-diagonal energy of the v-th row of W^r and decreases by a factor of $1/T$ when averaging over T independent Rademacher probes.

Proof (Proof of Lemma 2). Let $M = W^r$. Since z is a Rademacher probe vector, its entries are independent and satisfy $\mathbb{P}(z_u = 1) = \mathbb{P}(z_u = -1) = 1/2$, $\mathbb{E}[z_u] = 0$, and $\mathbb{E}[z_u^2] = 1$. For a fixed node v, we have $Y_r(v) = z_v(Mz)_v = z_v \sum_{u\in V} M_{vu} z_u = \sum_{u\in V} M_{vu} z_v z_u$. Taking expectation gives $\mathbb{E}[Y_r(v)] = \sum_{u\in V} M_{vu}\mathbb{E}[z_v z_u]$.

If $u \neq v$, then z_v and z_u are independent and centered, so $\mathbb{E}[z_v z_u] = 0$. If $u = v$, then $z_v z_u = z_v^2 = 1$. Hence $\mathbb{E}[Y_r(v)] = M_{vv} = (W^r)_{vv} = c_r(v)$.

It remains to compute the variance. From the expansion above, $Y_r(v) = M_{vv} + \sum_{u\neq v} M_{vu} z_v z_u$. Therefore, $Y_r(v) - \mathbb{E}[Y_r(v)] = \sum_{u\neq v} M_{vu} z_v z_u$. Thus, $\mathrm{Var}(Y_r(v)) = \mathbb{E}\left[\left(\sum_{u\neq v} M_{vu} z_v z_u\right)^2\right]$. Expanding the square yields

$$\mathrm{Var}(Y_r(v)) = \sum_{u\neq v}\sum_{w\neq v} M_{vu} M_{vw}\mathbb{E}[z_v z_u z_v z_w].$$

Since $z_v^2 = 1$, we have $\mathbb{E}[z_v z_u z_v z_w] = \mathbb{E}[z_u z_w]$. If $u \neq w$, then $\mathbb{E}[z_u z_w] = 0$; if $u = w$, then $\mathbb{E}[z_u^2] = 1$. Hence all cross terms vanish and only the diagonal terms remain. Therefore, $\mathrm{Var}(Y_r(v)) = \sum_{u\neq v} M_{vu}^2$. Now let $z^{(1)}, \ldots, z^{(T)}$ be T independent Rademacher probes and let $Y_r^{(t)}(v) = z_v^{(t)}(Mz^{(t)})_v$. Since the probes are independent, $Y_r^{(1)}(v), \ldots, Y_r^{(T)}(v)$ are independent and identically distributed. Therefore, $\mathbb{E}[\widehat{c}_r(v)] = \frac{1}{T}\sum_{t=1}^{T}\mathbb{E}[Y_r^{(t)}(v)] = M_{vv} = (W^r)_{vv}$. Similarly,

$$\mathrm{Var}(\widehat{c}_r(v)) = \mathrm{Var}\left(\frac{1}{T}\sum_{t=1}^{T} Y_r^{(t)}(v)\right) = \frac{1}{T^2}\sum_{t=1}^{T}\mathrm{Var}(Y_r^{(t)}(v)) = \frac{1}{T}\sum_{u\neq v} M_{vu}^2.$$

Substituting $M = W^r$ gives $\mathrm{Var}(\widehat{c}_r(v)) = \frac{1}{T}\sum_{u\neq v}(W^r)_{vu}^2$. Since the expectation identity holds for every node v, we also have $\mathbb{E}[\widehat{\mathbf{c}}_r] = \mathrm{Diag}(W^r)$.

Proof (Proof of Lemma 3). Consider one sampled prefix $\pi = (v_0, v_1, \ldots, v_{r-1})$ with $v_0 = v$. If the prefix reaches a node with zero weighted out-degree before

completing the required length, the sample is terminated and $X_r(v;\pi) = 0$, so the desired bound holds.

Otherwise, every sampled transition satisfies $W_{v_i v_{i+1}} > 0$. Under the locally normalized proposal, we have $Q(v_i, v_{i+1}) = \frac{W_{v_i v_{i+1}}}{d_W(v_i)}$. Hence the single-sample estimator satisfies

$$X_r(v;\pi) = \frac{\left(\prod_{i=0}^{r-2} W_{v_i v_{i+1}}\right) W_{v_{r-1} v}}{\prod_{i=0}^{r-2} Q(v_i, v_{i+1})} = \left(\prod_{i=0}^{r-2} d_W(v_i)\right) W_{v_{r-1} v}.$$

Since W is nonnegative, $X_r(v;\pi) \geq 0$. Moreover, by the definitions of $d_{\max}$ and $w_{\max}$, we have $d_W(v_i) \leq d_{\max}$ for every i and $W_{v_{r-1} v} \leq w_{\max}$. Therefore, $0 \leq X_r(v;\pi) \leq d_{\max}^{r-1} w_{\max}$. Thus the independent samples $X_r(v;\pi^{(1)}), \ldots, X_r(v;\pi^{(s)})$ are bounded in $[0, d_{\max}^{r-1} w_{\max}]$. Since $\widehat{c}_r(v) = \frac{1}{s}\sum_{j=1}^{s} X_r(v;\pi^{(j)})$ and $\mathbb{E}[\widehat{c}_r(v)] = c_r(v)$ by Lemma 1, Hoeffding's inequality gives

$$\Pr\left(|\widehat{c}_r(v) - c_r(v)| \geq \epsilon\right) \leq 2\exp\left(-\frac{2s\epsilon^2}{\left(d_{\max}^{r-1} w_{\max}\right)^2}\right).$$

Equivalently, $\Pr\left(|\widehat{c}_r(v) - c_r(v)| \geq \epsilon\right) \leq 2\exp\left(-\frac{2s\epsilon^2}{d_{\max}^{2r-2} w_{\max}^2}\right)$.

Proof (Proof of the Lemma 4). Let $M = W^r$ and $Y_r(v) = z_v (Mz)_v$. Since $z_u \in \{-1, +1\}$ and $W \geq 0$, we have $|Y_r(v)| \leq \sum_{u \in V} (W^r)_{vu} \leq d_{\max}^r$. Thus $Y_r(v) \in [-d_{\max}^r, d_{\max}^r]$. Since $\widehat{c}_r(v)$ is the average of T independent copies of $Y_r(v)$ and $\mathbb{E}[\widehat{c}_r(v)] = c_r(v)$, the result follows from Hoeffding's inequality.

Proof (Proof of the Theorem 1). For every node $v \in V_{\text{low}}$, Estimator 1 computes the exact value, so $\widetilde{c}_r(v) = c_r(v)$ and the approximation error is zero. For every $v \in V_{\text{high}}$ and every $r \in \{2, \ldots, q\}$, Lemma 3 gives $\Pr\left(|\widehat{c}_r(v) - c_r(v)| \geq \epsilon\right) \leq 2\exp\left(-\frac{2s\epsilon^2}{(M_{\max}^{\text{IS}})^2}\right)$. There are $|V_{\text{high}}|(q-1)$ stochastic node-order pairs. Applying the union bound gives $\Pr\Big(\exists v \in V_{\text{high}},\, \exists r \in \{2, \ldots, q\} : |\widehat{c}_r(v) - c_r(v)| \geq \epsilon\Big) \leq 2|V_{\text{high}}|(q-1)\exp\Big(-\frac{2s\epsilon^2}{(M_{\max}^{\text{IS}})^2}\Big)$. Combining this with the zero-error property on V_{low} proves the theorem.

Proof (Proof of the Theorem 2). For every node $v \in V_{\text{exact}}$, the hybrid estimator replaces the stochastic estimate by the exact value, so $\widetilde{c}_r(v) = c_r(v)$ and the error is zero. For every $v \in V_{\text{stoch}}$ and every $r \in \{2, \ldots, q\}$, Lemma 4 gives

$$\Pr\left(|\widehat{c}_r(v) - c_r(v)| \geq \epsilon\right) \leq 2\exp\left(-\frac{T\epsilon^2}{2(M_{\max}^{\text{Vec}})^2}\right).$$

There are $|V_{\text{stoch}}|(q-1)$ stochastic node-order pairs. Applying the union bound, we obtain $\Pr\Big(\exists v \in V_{\text{stoch}},\, \exists r \in \{2, \ldots, q\} : |\widehat{c}_r(v) - c_r(v)| \geq \epsilon\Big) \leq 2|V_{\text{stoch}}|(q-1)\exp\Big(-\frac{T\epsilon^2}{2(M_{\max}^{\text{Vec}})^2}\Big)$. The lower bound on T makes the right-hand side at most δ. Therefore, with probability at least $1-\delta$, all stochastic HISP-Vec features on V_{stoch} are within ϵ of their exact values. Together with the zero-error property on V_{exact}, this proves the claim.

GNN-based Influence Estimation with HISP Features

This section incorporates HISP features into a GNN-based influence estimator. The feature matrix $\widetilde{\mathbf{C}}$, produced by the Estimators 1 or 2, approximates the exact closed-walk statistics $c_r(v) = (W^r)_{vv}$.

- *Training data and node features.* For each training graph and seed set S, offline Monte Carlo simulations generate cumulative infection probability vectors $\pi_0, \pi_1, \ldots, \pi_h$, where $\pi_i(v) = \Pr(v \text{ is activated by step } i \mid S)$. The initial vector π_0 is the seed indicator. The GNN is trained to predict the next-step infection probability from previous diffusion states and HISP structural features. Let b be the history length. We use zero padding for unavailable history states, namely $\pi_j = \mathbf{0}$ for all $j < 0$. For diffusion step $i \geq 1$, the initial feature vector of node v is $h_v^{(0)} = [\pi_{i-b+1}(v) - \pi_{i-b}(v), \ldots, \pi_{i-1}(v) - \pi_{i-2}(v), \pi_{i-1}(v), \widetilde{c}_2(v), \ldots, \widetilde{c}_q(v)]$.

- *Diffusion-aware message passing.* The structural matrix W is used to construct the HISP closed-walk features. For message passing, we use a possibly different diffusion-weight matrix Ω. Under the IC model, $\Omega = P$. Under the MONSTOR+-style LT setting, we use the normalized incoming-neighbor weight $\Omega_{uv} = \begin{cases} 1/|N^-(v)|, & (u,v) \in E, \\ 0, & \text{otherwise} \end{cases}$. This separates the structural matrix W from the message-passing matrix Ω. For each GNN layer $\ell = 1, \ldots, L$, the message from $u \in N^-(v)$ to v is $m_{u\to v}^{(\ell)} = \Omega_{uv} \left(h_u^{(\ell-1)} \Theta_1^{(\ell)} + b_1^{(\ell)} \right)$. Let $\mathcal{M}_v^{(\ell)} = \{m_{u\to v}^{(\ell)} : u \in N^-(v)\}$. The aggregated message is

$$a_v^{(\ell)} = \mathrm{CONCAT}\left(\mathrm{SUM}(\mathcal{M}_v^{(\ell)}), \mathrm{MEAN}(\mathcal{M}_v^{(\ell)}), \mathrm{MAX}(\mathcal{M}_v^{(\ell)}), \mathrm{STD}(\mathcal{M}_v^{(\ell)}) \right).$$

The node representation is updated by $h_v^{(\ell)} = \mathrm{ReLU}\left([h_v^{(\ell-1)} \| a_v^{(\ell)}] \Theta_2^{(\ell)} \right)$.

- *Monotone step prediction.* To preserve the cumulative nature of infection probabilities, the model computes $z_v = g_\theta(h_v^{(L)})$ and predicts

$$\widehat{\pi}_i(v) = \pi_{i-1}(v) + (1 - \pi_{i-1}(v))\, \sigma(z_v), \tag{4}$$

where $\sigma(\cdot)$ is the sigmoid function.

Proposition 1 (Temporal monotonicity). *If $\pi_{i-1}(v) \in [0,1]$, then Eq. (4) satisfies $\pi_{i-1}(v) \leq \widehat{\pi}_i(v) \leq 1$.*

Proof. Since $0 < \sigma(z_v) < 1$, $0 \leq (1 - \pi_{i-1}(v))\, \sigma(z_v) \leq 1 - \pi_{i-1}(v)$. Adding $\pi_{i-1}(v)$ to all sides gives the claim.

This monotonicity is temporal, namely across diffusion steps. It does not imply monotonicity or submodularity of the learned set function $\widehat{\sigma}_\theta(S)$ with respect to the seed set S.

The model is trained by minimizing $\mathcal{L}_{\mathrm{IE}} = \frac{1}{|\mathcal{T}|} \sum_{(G,S,i)\in\mathcal{T}} \frac{1}{n} \left\| \widehat{\pi}_i^{G,S} - \pi_i^{G,S} \right\|_1$, where $\mathcal{T}$ is the set of training tuples. At inference time, the trained one-step model is iteratively applied s_{stack} times from π_0, yielding the final infection probability estimate $\widehat{\pi}$. The estimated influence of seed set S is $\widehat{\sigma}_\theta(S) = \mathbf{1}^\top \widehat{\pi}$.

Algorithm 3: HISP-GREEDY: Greedy Seed Selection with a Trained GNN Oracle

Input: Graph $G = (V, E)$, HISP features $\widetilde{\mathbf{C}}$, trained GNN influence estimator F_θ, seed budget k
Output: Seed set S

$S \leftarrow \emptyset$;
for $t \leftarrow 1$ **to** k **do**
  foreach $v \in V \setminus S$ **do**
    Compute $\widehat{\Delta}_\theta(v \mid S) \leftarrow F_\theta(G, S \cup \{v\}, \widetilde{\mathbf{C}}) - F_\theta(G, S, \widetilde{\mathbf{C}})$;
  $v_t \leftarrow \arg\max_{v \in V \setminus S} \widehat{\Delta}_\theta(v \mid S)$;
  $S \leftarrow S \cup \{v_t\}$;
return S;

Learned-Oracle Influence Maximization

Given the trained GNN influence estimator F_θ, HISP uses it as a fast surrogate oracle for seed selection. We write $F_\theta(G, S, \widetilde{\mathbf{C}}) = \widehat{\sigma}_\theta(S)$. For a current seed set $S \subseteq V$ and a candidate $v \notin S$, define the learned marginal gain $\widehat{\Delta}_\theta(v \mid S) = F_\theta(G, S \cup \{v\}, \widetilde{\mathbf{C}}) - F_\theta(G, S, \widetilde{\mathbf{C}})$. Starting from $S_0 = \emptyset$, the greedy procedure selects $v_t \in \arg\max_{v \in V \setminus S_{t-1}} \widehat{\Delta}_\theta(v \mid S_{t-1})$, and sets $S_t = S_{t-1} \cup \{v_t\}$ for $t = 1, \ldots, k$. Since F_θ is not guaranteed to be submodular, Algorithm 3 recomputes all candidate marginal gains at each iteration. A CELF-style implementation may be used heuristically, although its lazy upper-bound interpretation is not guaranteed for a non-submodular learned oracle.
Under the standard IC and LT models, the ground-truth influence function σ is monotone and submodular [6], whereas the learned oracle F_θ need not be. Nevertheless, if the learned marginal gains uniformly approximate the true marginal gains, greedy selection admits an additive approximation guarantee with respect to σ.

Overall Framework

Algorithm 4 summarizes the complete HISP-IM pipeline. The training phase learns the GNN influence estimator from offline Monte Carlo diffusion trajectories and HISP structural features. The inference phase constructs HISP features for the target graph and runs Algorithm 3 with the trained GNN oracle.

Detailed experimental results

Experimental setup. We evaluate HISP under the LT setting on three directed social-network datasets used in [7]: Extended, Celebrity, and WannaCry; their statistics are summarized in Table 1. We compare three methods: **MONSTOR+**, **HISP-I** and **HISP-Vec** ***For feature accuracy***, HISP-I uses $s \in \{20, 50, 100\}$ samples per node, while HISP-Vec uses $T = 1000$ probes. For preprocessing cost and for the influence-maximization experiment, HISP-I uses $s = 1000$. As in [7], we set $q = 11$. In addition, we set $\tau_d = 0$, so that no node with positive out-degree

Algorithm 4: HISP-IM: Complete Framework

Input: Training graphs $\mathcal{G}_{\text{train}}$, target graph G, diffusion model, seed budget k, structural order q, and HISP parameters
Output: Seed set S
// Training phase
foreach $G' \in \mathcal{G}_{\text{train}}$ **do**
  Construct the model-specific structural matrix $W_{G'}$;
  Compute HISP features $\widetilde{\mathbf{C}}_{G'}$ using Algorithm 1 or Algorithm 2;
  Generate training diffusion trajectories by offline Monte Carlo simulation;
Train the GNN influence estimator F_θ using the generated trajectories and HISP features;
// Inference and seed-selection phase
Construct the model-specific structural matrix W_G;
Compute HISP features $\widetilde{\mathbf{C}}_G$ using Algorithm 1 or Algorithm 2;
$S \leftarrow$ Algorithm 3 $(G, \widetilde{\mathbf{C}}_G, F_\theta, k)$;
return S;

is computed exactly; both estimators are thus evaluated over the entire dataset to assess their overall effectiveness.

For preprocessing efficiency, for MONSTOR+, we report two exact implementations: **MONSTOR+-Full**, which computes the structural features from full matrix powers, and **MONSTOR+-Block**, our implementation of the block-wise scheme of MONSTOR+ (blocks of 2000 columns), which computes the same exact features to reduce peak memory. We address three questions: **Q1:** how accurately HISP approximates the exact structural features; **Q2:** how much preprocessing time and memory it saves over MONSTOR+; and **Q3:** how approximate HISP features affect downstream GNN influence-estimation accuracy. All methods use the same graph representation and target the same closed-walk features. The feature-accuracy and preprocessing-cost experiments were run on a Google Colab instance with an Intel Xeon CPU @ 2.20GHz (2 cores) and 12GB RAM. GNN training and influence maximization were run on a separate workstation described in the Appendix. The graphs and the exact features are stored in double precision. HISP-Vec sets the features of nodes that lie on no closed walk to exactly zero. These nodes are found by repeatedly removing nodes whose in-degree or out-degree is zero. HISP-Vec does not clip negative estimates, so the estimator stays unbiased. HISP-I reuses each sampled prefix for all orders $r = 2, \ldots, q$. The same settings are used when computing the features for GNN training.

Times are single-run wall-clock measurements with random seed 0, and graph loading is not timed. Peak memory is the maximum memory allocated for the data produced during the computation (arrays and sparse matrices), measured with Python's tracemalloc. It excludes the input graph and the memory of the running process itself, such as the interpreter and the loaded libraries, so it is not the total memory used by the program. Temporary buffers inside compiled library routines are not tracked. Because tracemalloc was enabled during timing, the reported times include its overhead, and the HISP-I times exclude building the sampler.

Table 1: Statistics of the datasets used in the experiments.

Dataset	#Nodes (n)	#Edges (m)
Extended [7]	5,413	27,146
Celebrity [7]	7,336	27,699
WannaCry [7]	19,381	85,202

Table 2: Structural-feature approximation accuracy under LT.

Dataset	Method	RMSE	Pearson	Spearman
Extended	Exact	0.0000	1.000	1.000
	HISP-I (20)	1.373×10^{-1}	0.635	0.538
	HISP-I (50)	1.108×10^{-1}	0.707	0.601
	HISP-I (100)	1.170×10^{-1}	0.694	0.648
	HISP-Vec (1000)	1.398×10^{-2}	0.995	0.918
Celebrity	Exact	0.0000	1.000	1.000
	HISP-I (20)	5.926×10^{-3}	0.702	0.537
	HISP-I (50)	3.443×10^{-2}	0.255	0.599
	HISP-I (100)	5.301×10^{-2}	0.225	0.643
	HISP-Vec (1000)	7.555×10^{-3}	0.695	0.327
WannaCry	Exact	0.0000	1.000	1.000
	HISP-I (20)	2.272×10^{-1}	0.487	0.493
	HISP-I (50)	1.353×10^{-1}	0.713	0.539
	HISP-I (100)	1.215×10^{-1}	0.749	0.575
	HISP-Vec (1000)	1.423×10^{-2}	0.996	0.726

Table 3: Structural preprocessing cost under LT (single run).

Dataset	Method	Time (s)	Mem. (MB)	Speedup
Extended	Full	0.80	36.0	1.00×
	Block	5.08	13.8	0.16×
	HISP-I (1000)	378.46	0.5	0.002×
	HISP-Vec (1000)	0.51	11.5	1.55×
Celebrity	Full	4.38	248.7	1.00×
	Block	9.58	68.5	0.46×
	HISP-I (1000)	419.76	0.7	0.01×
	HISP-Vec (1000)	0.91	15.6	4.81×
WannaCry	Full	24.67	1461.1	1.00×
	Block	26.26	156.6	0.94×
	HISP-I (1000)	1322.99	1.7	0.02×
	HISP-Vec (1000)	3.34	41.2	7.39×

Evaluation Metrics. We evaluate HISP from three complementary perspectives: structural-feature accuracy, preprocessing efficiency, and downstream influence-estimation quality. Structural accuracy measures how closely the estimated closed-walk features match their exact counterparts, preprocessing efficiency quantifies the computational gain over exact MONSTOR+ preprocessing, and downstream accuracy evaluates the effect of approximate structural features on influence estimation. The evaluation metrics are defined as follows: (1) Feature RMSE: $\mathrm{RMSE}_C = \left(\frac{1}{n(q-1)}\sum_{v\in V}\sum_{r=2}^{q}(\widetilde{c}_r(v)-c_r(v))^2\right)^{1/2}$; (2) Preprocessing speedup: $\mathrm{Speedup} = \frac{T_{\mathrm{pre}}(\mathrm{MONSTOR+\text{-}Exact})}{T_{\mathrm{pre}}(\mathrm{HISP})}$; (3) Influence-estimation MAE: $\mathrm{MAE}_\sigma = \frac{1}{|\mathcal{D}_{\mathrm{test}}|}\sum_{S\in\mathcal{D}_{\mathrm{test}}}|\widehat{\sigma}_\theta(S)-\sigma_{\mathrm{MC}}(S)|$. Peak memory, reported together with the preprocessing time, is the maximum memory allocated for the data produced during the feature computation, namely arrays and sparse matrices. It is not the memory footprint of the running process.

Results and Discussion. Table 2 and Table 3 show the results. Here, "Full" denotes MONSTOR+'s full matrix-power implementation, whereas "Block" denotes its memory-efficient block-wise exact implementation. For HISP-I and HISP-Vec, the numbers in parentheses denote s and T, respectively. Table 2 shows that HISP-Vec provides the most accurate approximation of the exact structural features, achieving an average RMSE of 0.012 and Pearson correlation of 0.895, compared with approximately 0.097 and 0.556, respectively, for HISP-I ($s = 100$). In particular, HISP-Vec reaches Pearson correlations above 0.99 on Extended

and WannaCry, indicating that it closely preserves the exact closed-walk feature structure. Regarding time and memory, the results are shows in Table 3. Table 3 further shows how the cost of HISP-Vec scales over the three evaluated networks (up to $n = 19{,}381$). It is the only approximate method that consistently accelerates preprocessing, with speedup increasing from 1.55× on Extended to 7.39× on WannaCry and averaging 4.58×. Its average peak memory is only 22.8 MB, compared with 581.9 MB for full matrix-power computation. HISP-I achieves the lowest memory usage (at most 1.7 MB) but is 54 to 473 times slower than MONSTOR+-Full in our implementation, while MONSTOR+-Block reduces memory at the cost of additional runtime. Overall, among the evaluated methods, HISP-Vec provides the best balance between approximation accuracy, runtime, and memory efficiency.

HISP-I versus HISP-Vec. On our evaluation graphs, HISP-I is not the more accurate estimator. HISP-Vec has lower RMSE and higher Pearson correlation than HISP-I at every tested $s \leq 100$ on Extended and WannaCry, and HISP-I is competitive only on Celebrity at small s. This agrees with the variance behavior noted for Estimator 1, which can be large when closed-walk returns are rare. These networks are sparse, with an average degree of about five or less, so sampled walks likely seldom return. The accuracy of HISP-I is also not monotone in s in our single runs. On Celebrity, for example, the RMSE grows from 5.9×10^{-3} at $s = 20$ to 5.3×10^{-2} at $s = 100$. We do not claim a cause for this. The runtime gap points to our implementation rather than to the operation count. With $s = T = 1000$ and $q = 11$, HISP-I needs at most about 6.0×10^7 sampling steps on Extended, while HISP-Vec needs about 3.0×10^8 elementary operations. Yet in our single runs the sampler is about 742 times slower, which means it processes roughly 3,700 times fewer steps per second. We expect an optimized sampler to narrow this gap. A comparison on denser and more clustered graphs, where returns are frequent, is left to future work.

Limitations. Feature accuracy and cost are measured on three networks with at most 19,381 nodes, from a single run per configuration, so we report no variability across random seeds. The downstream comparison covers only the Extended network under the LT model, with one GNN per feature type evaluated at its first-epoch checkpoint, trained on a fixed random subset of the 130 synthetic graphs (eight for MONSTOR+, fifteen for HISP-I and HISP-Vec), and we do not analyze the large drop of HISP-Vec at $k = 10$. HISP-I is 54 to 473 times slower than MONSTOR+-Full in our implementation, so its advantage here is limited to memory. Our analysis bounds the error of the structural features, but we do not prove that this error implies the uniform marginal-gain bound assumed in Proposition 2, so the greedy guarantee remains conditional. Finally, both estimators are applied to the entire graph without the τ_d partition, so the benefit of the hybrid design is not evaluated.

Additional experiments

We train the GNN on synthetic directed graphs generated by the Barabási–Albert [1] and Holme–Kim [5] models, following Panagopoulos et al. [11]. The pool contains 100 graphs with 100–200 nodes and 30 graphs with 300–500 nodes, with weighted-cascade edge probabilities $p_{(u,v)} = 1/\deg(v)$ [6]. Training labels are generated by 1K Monte Carlo simulations for each of 4K seed sets per graph (2K from each of the two seed-sampling schemes of MONSTOR+). Because all training data are loaded into the 15 GB of host memory at once, each GNN uses a random subset of the pool, drawn once with Python's `random.Random(42).sample` over the sorted graph names: eight graphs for MONSTOR+ (four with 300–500 nodes and four with 100–200 nodes) and fifteen for HISP-I and HISP-Vec (seven and eight), the eight being contained in the fifteen. The selected graph names are listed in `TRAINING_GRAPHS.md` in the released code. A separate GNN is trained for each feature type (exact for MONSTOR+, HISP-I, HISP-Vec); we do not train one model and swap features at evaluation time. All models share the same architecture (three layers, hidden dimension 32, input dimension 4 plus the $q - 1 = 10$ structural features), the Adam optimizer with mini-batches of 50 training tuples, a learning rate of $10^{-4} \times$ epoch for the first nine epochs and 10^{-2}/epoch afterwards, 100 epochs, and random seed 42. The weights of the graph-convolution layers use Xavier uniform initialization with the ReLU gain. The structural features are fed to the network as computed, cast to single precision, without normalization, clipping, or dropout. The loss is the squared error between predicted and Monte Carlo infection probabilities, averaged over nodes and summed over the mini-batch, and the optional regularizers are not used. For each graph, 3,200 of its 4,000 seed sets are used for training (1,600 per sampling scheme) and the remaining 800 for validation, so validation uses held-out seed sets of the training graphs; there is no split across graphs, and the real networks are used only for testing. The checkpoint used below is the one saved after the first epoch, because our validation criterion did not distinguish between epochs. Training and inference ran on a workstation with an NVIDIA GeForce GTX 1050 Ti GPU (4 GB) and 15 GB of RAM. We then compare MONSTOR+ with HISP-I and HISP-Vec on the Extended dataset [7] in terms of the final IM objective value, with $q = 11$; HISP-I uses $s = 1000$ and HISP-Vec uses $T = 1000$. The results are reported below.

Influence-maximization performance. We further evaluate seed set selection quality on the Extended dataset using 100,000 independent Monte Carlo simulations (Table 4). For seed budgets $k = 10, \ldots, 100$, both estimators trail MONSTOR+ at small budgets, most severely HISP-Vec at $k = 10$ (-51%) and HISP-I (-12%); the gap narrows to 2–3% by $k = 20$. From $k = 30$ onward, HISP-I consistently exceeds MONSTOR+ by 11–17%, while HISP-Vec turns positive from $k = 40$ onward and stabilizes at a smaller 2–7% margin. HISP-I's advantage over MONSTOR+ is markedly larger than HISP-Vec's, even though HISP-Vec attains lower RMSE and higher Pearson correlation than HISP-I at every tested $s \leq 100$ on Extended and WannaCry (Table 2). Feature accuracy at the downstream setting

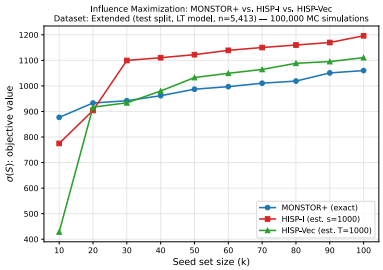


Fig. 2: Influence Maximization objective value comparison

Table 4: Influence $\sigma_{\mathrm{MC}}(S)$ of seed sets selected by CELF with each learned oracle on Extended (LT, 100,000 Monte Carlo simulations, single run). The oracles are first-epoch checkpoints, trained on eight graphs for MONSTOR+ and fifteen for HISP-I and HISP-Vec. Relative change with respect to MONSTOR+ is in parentheses.

k	MONSTOR+	HISP-I	HISP-Vec
10	877.0	774.9 (−11.6%)	428.4 (−51.1%)
20	933.2	904.1 (−3.1%)	917.0 (−1.7%)
30	941.9	1,099.6 (+16.7%)	933.9 (−0.9%)
40	961.7	1,110.4 (+15.5%)	980.0 (+1.9%)
50	987.1	1,122.2 (+13.7%)	1,032.7 (+4.6%)
60	997.2	1,139.3 (+14.2%)	1,049.4 (+5.2%)
70	1,010.6	1,150.2 (+13.8%)	1,064.4 (+5.3%)
80	1,018.9	1,160.0 (+13.8%)	1,088.6 (+6.8%)
90	1,050.9	1,169.8 (+11.3%)	1,095.2 (+4.2%)
100	1,060.0	1,196.2 (+12.8%)	1,110.7 (+4.8%)

$s = 1000$ was not measured, so we do not attribute the difference to feature accuracy. Since the HISP models were trained on more graphs than MONSTOR+ and all checkpoints come from the first epoch, we treat the comparison with MONSTOR+ as exploratory and draw no conclusion about the effect of approximate features on downstream quality. Their estimation accuracy is limited Table 5, so these results do not establish the quality of the oracles themselves.

Influence-estimation accuracy. Table 5 reports the mean absolute error (MAE) between the total influence predicted by each oracle and the Monte Carlo influence of the 4,000 test seed sets of Extended. The three oracles have almost the same error, within 0.7% of each other, so replacing the exact features by the HISP estimates does not change the estimation accuracy of the trained oracle. However, none of them estimates influence accurately. Their predictions average between 95 and 105 against a true mean of 1,298, and their MAE is only about 3% below that of a baseline that predicts the number of seeds. The correlation of their predictions with the true influence is also lower than that of the number of seeds

Table 5: Error of the estimated total influence on the 4,000 test seed sets of Extended (LT, first-epoch checkpoints, $s = 1$); the true influence averages 251 and 2,345 in the first and second half of the seed sets. "No spread" predicts the number of seeds.

Oracle	MAE	MAE (1st half)	MAE (2nd half)	Pearson	Spearman
MONSTOR+	1,209.7	164.5	2,255.0	0.193	0.324
HISP-I	1,202.2	159.3	2,245.1	0.187	0.319
HISP-Vec	1,207.8	162.9	2,252.7	0.178	0.312
No spread	1,244.3	197.7	2,291.0	0.401	0.499

alone. For MONSTOR+, the checkpoint of epoch 100 gives predictions identical to those of epoch 1 on Extended. We therefore regard these oracles as weak and read Table 4 as exploratory.